Beyond the Chosen Token: Detecting LLM Hallucinations from Full Output Distributions

Final Project

Course Name: Deep Learning on Computational Accelerators

Course Number: 02360781

Year: 2025-2026 Semester: Winter

Lecturer: Prof. Or Litany

Teaching Assistants: Amir Man (Head), Moshe Kimhi, Itay Elem

Students: Natan Izhak Poor, Abed Semre

ID Numbers: 211412986, 214682353

Submission Date: 15/03/2026

Abstract

Large language models (LLMs) often generate hallucinated information, making reliable hallucination detection a critical research problem. In this project, we reproduce the LOS-Net framework, a gray-box approach proposed for detecting hallucinations using the complete sequence of LLM output distributions [1]. We rigorously replicate the original experimental setup across multiple LLM and dataset combinations, validating that the baseline architecture achieves results consistent with the original study.

Building on this reproduction, we introduce several efficiency and experimental improvements. Most notably, we redesign the data loading pipeline using a Direct-to-VRAM initialization scheme that eliminates repeated disk transfers during training, and we implement highly optimized GPU inference routines utilizing TensorFloat-32 (TF32) precision. These system-level optimizations drastically accelerate the computational pipeline, enabling us to execute an exhaustive 1,800-run randomized hyperparameter search that was computationally infeasible under the original methodology.

Across the nine evaluated model–dataset combinations, our approach accelerates training time in **8 settings** (**with 5 of them being clear improvements**), reduces detection latency in **all 9 settings** (**with 3 clear improvements**), and improves predictive performance over the original paper in **7 settings** (**5 clear improvements**), achieving absolute gains of up to **+2.89 AUC**. Ultimately, our results demonstrate that resolving fundamental hardware–software bottlenecks not only accelerates experimentation but also unlocks substantially stronger predictive configurations.

Introduction

Large Language Models (LLMs), powered by deep neural networks and transformer architectures, have become a central technology in artificial intelligence, proving that *attention is all you need* to achieve remarkable text generation capabilities [2]. However, their widespread adoption is hindered by a critical flaw: the tendency to produce confident-sounding text that contains factually incorrect statements or fabricated information, commonly known as hallucinations. The automated detection of these hallucinations is pivotal

to ensuring the safe and reliable deployment of LLMs in real-world scenarios, particularly in high-stakes domains where outputs are acted upon without verification [3].

Prior work has proposed a variety of approaches to hallucination detection, ranging from white-box methods that require access to internal model representations [4], to gray-box methods that operate on observable decoding outputs such as token-level log-probabilities [5]. Gray-box approaches are of particular practical importance because they remain applicable to closed-source models exposed only through restricted APIs. However, many existing gray-box techniques, such as Semantic Entropy, primarily rely on scalar statistics derived from the log-probabilities assigned to the generated tokens during decoding, which we refer to as Actual Token Probabilities (ATP) [6]. This perspective overlooks the broader information contained in the full next-token probability distributions over the vocabulary at each generation step, which we term the Token Distribution Sequence (TDS). Importantly, the ATP value alone can be misleading. For example, a token selected from a highly confident prediction and one sampled from a broad, uncertain distribution might have the same ATP, even though the model's underlying uncertainty is very different. This hides patterns in the full token distribution that could indicate whether the output is correct.

To bridge this gap, Bar-Shalom et al. [1] proposed learning directly from the complete observable output of the LLM. They introduced the LLM Output Signature (LOS), a unified representation pairing the TDS and ATP. Building on this, they developed LOS-Net, a lightweight, attention-based architecture trained on an efficient encoding of the LOS. LOS-Net retains only the top-K probabilities at each step and applies a Rank Encoding mechanism to create a standardized representation that contextualizes each token probability relative to overall model confidence. This approach achieves superior hallucination detection performance while maintaining a detection latency of approximately 10^{-5} seconds per instance, orders of magnitude faster than baseline methods.

In this project, we aim to reproduce the hallucination detection results of the original paper across the evaluated datasets (HotpotQA, IMDB, Movies) and target LLMs (Llama, Mistral, Qwen) to assess the stability of the approach.

Furthermore, our work extends beyond strict reproduction to address critical computational inefficiencies in the baseline architecture. Upon identifying a severe I/O bottleneck caused by lazy disk loading, we engineered a Direct-to-VRAM initialization pipeline that reduces training epoch times from minutes to seconds. We complement this with software-level inference optimizations, including strict GPU synchronization and TensorFloat-32 (TF32) acceleration, to achieve single-digit microsecond detection latency. Finally, we leverage this highly scalable framework to conduct vastly expanded hyperparameter sweeps, comprehensive cross-domain transferability evaluations, and a methodologically corrected ablation study on vocabulary window restrictions. Our modifications yielded significant improvements in both computational efficiency and downstream performance. Ultimately, this project demonstrates that state-of-the-art hallucination detection can be made both highly robust and exceptionally efficient.

Note: The official repository of this project is:
https://github.com/NatiTechnion/los-net-reproduction

Reproduction

Setup and Hardware

Our first objective was to reproduce the results of the original paper by following the instructions provided in the README.md file in the official Git repository. We initially attempted to run the full pipeline on the course server. After installing the required dependencies and creating a Conda environment, our first task was to generate the datasets using the LLMs.

After running the generation process for some time, it became clear that the course server was not sufficient for our needs. The hardware limitations, including restricted VRAM, low-end GPUs, and limited storage, made it impractical to complete all the required computations. After several attempts to optimize the setup, we decided to migrate to a dedicated cloud server hosted on RunPod (https://www.runpod.io).

We are aware that some groups addressed hardware constraints by reduc-

ing the dataset size, limiting the hyperparameter search space, or applying techniques such as data quantization. Although we initially considered these alternatives, we ultimately decided against them because they could potentially compromise performance and affect the fidelity of the reproduction.

Instead, we provisioned a high-performance server equipped with an NVIDIA H200 SXM GPU with 141GB of VRAM and an AMD EPYC CPU, along with ample RAM and storage. This configuration satisfied the computational requirements of the project. **In total, the cost of the computational resources exceeded 15,000 NIS, which we paid out of our own pocket.**

Data Generation and Preprocessing

To generate the datasets, we used the data generation script included in the Git repository (`scripts/generate_HD_raw_datasets.sh`) on the LLMs supported for hallucination detection: Llama, Mistral, and Qwen. This step produced the raw dataset files (the `.pt` files) containing the model outputs required for subsequent preprocessing and training.

The next step was to preprocess the raw datasets. For this, we used the provided `scripts/preprocess_raw_datasets_LOS.sh` script, which we modified to fit our project needs, such as removing data contamination datasets and models that were not part of this work.

On our server, both generation and preprocessing ran smoothly. Although we encountered some initial technical difficulties, these were resolved quickly with ChatGPT. We were ultimately able to execute multiple jobs in parallel, taking full advantage of the available GPU memory and computational resources. Parallel execution significantly reduced the total generation and preprocessing time, enabling us to efficiently prepare all datasets required for the subsequent experiments. Overall, the data generation and preprocessing steps took several days to a week to complete. Notably, we performed these steps for all dataset/model combinations (nine in total), using the full data and full precision (i.e., without quantization or any other approximation methods). In addition, we conducted the transferability experiments and several additional experiments testing multiple top_k values as a bonus, which we will discuss later.

Training and Reproducing

The next phase of the project was to use the preprocessed data to train our architecture. To do this, we attempted to run the wandb sweeps as instructed in the original Git repository. However, we encountered several difficulties that caused significant delays. We initially allowed the sweeps to run for a few days, but soon realized that the process was taking far too long: a single run (not the full sweep, but just one set of hyperparameters) required several hours to complete, and sometimes even a full day. At this rate, completing all the necessary runs (there are over 3,000 of them!) would have taken well beyond the project deadline. We therefore understood that we needed to find a solution, which turned out to be a non-trivial task.

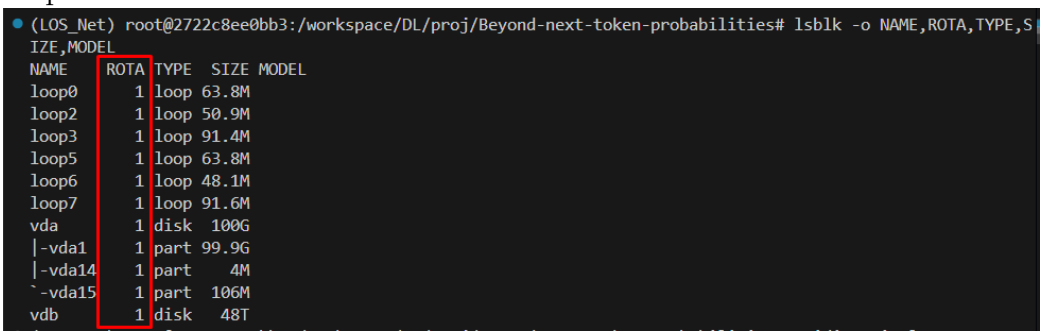
We obviously attempted to use various LLMs to fix the issue, including ChatGPT, Gemini, Claude, and Perplexity. We followed the recommendations they provided, for example, ensuring the code ran on the GPU, running multiple agents and jobs in parallel, and increasing `num_workers` to load data faster. However, we found that these suggestions did not resolve the issue, and in some cases the guidance was inaccurate or inconsistent with the problem we were trying to solve (some of them literally hallucinated while we are trying to fix this exact issue!).

At this point, we were stuck and created a Piazza post to ask for help: https://piazza.com/class/mgdhmltpzql7bg/post/144. At the same time, we decided to contact the author of the paper, Guy Bar-Shalom, via email. To our surprise, he responded and offered guidance. He suggested some basic debugging steps, such as verifying that the code was running on the GPU. We followed these steps and confirmed that the setup was correct.

When the issue persisted, Guy joined a Zoom call with us for further troubleshooting. During the one-hour session, he helped us understand how to perform timing tests to measure the duration of each step in the training process. These tests revealed the root cause of the slowdown: data loading. Most of the time in each epoch was being spent reading data from the disk rather than running model computations.

We investigated possible solutions with Guy, but ultimately determined that the disk on our server was a limiting factor, Guy said there's nothing we

can do unfortunately. After Guy left, we continued analyzing the setup and what disk we had using ChatGPT and Gemini, and confirmed our suspicion: our server used a rotational hard drive rather than an SSD!



We explored upgrading the disk to a high-end NVMe via RunPod, but such options were not available. We then returned to the dataset loading code to understand it more deeply. We discovered that the original implementation used lazy loading: at the start of each epoch, it would load only the data needed for that epoch from disk. While memory-efficient, this caused significant I/O overhead given our hardware. After several attempts to resolve the issue, primarily using ChatGPT, we began to suspect that there might be no practical solution, as was also suggested during our discussion with Guy. We prepared to reduce the grid search space and evaluate fewer hyperparameter configurations, as was later recommended in the Piazza response. However, the following day we had a brilliant idea: instead of trying to optimize around the bottleneck, we asked whether it would be possible to modify the data loading mechanism itself.

Specifically, we investigated whether we could replace the lazy loading strategy with a different architecture: loading the entire preprocessed dataset into RAM during initialization (i.e., at the start of each run), and only then begin the training process. Before modifying the code, we wanted to verify that this approach was feasible. We therefore examined the storage size of the preprocessed training datasets for each model, which is used to train the model. The results are shown in the following table:

Dataset / Model	Llama	Mistral	Qwen
movies	2GB	2GB	2GB
movies_test	1.6GB	1.6GB	1.6GB
imdb	2GB	2GB	2GB
imdb_test	2GB	2GB	2GB
hotpotqa	2GB	2GB	2GB
hotpotqa_test	2GB	2GB	2GB

Table 1: Dataset size per model (preprocessed data)

By the way, it was also cool to see that preprocessing significantly reduced the storage requirements of the datasets. Below are the storage sizes for each dataset–model combination before preprocessing (i.e., the raw data):

Dataset / Model	Llama	Mistral	Qwen
movies	229GB	102GB	247GB
movies_test	181GB	81GB	195GB
imdb	475GB	59GB	79GB
imdb_test	475GB	58GB	128GB
hotpotqa	299GB	102GB	363GB
hotpotqa_test	313GB	104GB	374GB

Table 2: Dataset size per model (raw data)

Since our system had significantly more RAM than required to train a single dataset/model combination (and we even had enough resources to run multiple jobs in parallel, as discussed later), we determined that loading the entire dataset into memory was feasible. After modifying the relevant code and resolving several implementation bugs, we tested the new approach by running a sweep to verify correctness and measure performance.

This optimization yielded a substantial improvement, reducing the runtime of a single training run by approximately one half. However, further timing experiments revealed that the solution was still not optimal. Although the dataset was now loaded entirely into RAM at initialization, each batch still had to be transferred from RAM to GPU memory (VRAM) during training (the original code transferred the data at the start of each epoch). These repeated transfers introduced a new bottleneck.

We therefore extended our approach: instead of loading the dataset into RAM, we modified the code again to now load the entire dataset directly into VRAM during the `__init__` phase. Based on our earlier storage analysis, we knew this was feasible given that we were using an NVIDIA H200 SXM GPU with 141GB of VRAM. We implemented the necessary changes to move the dataset tensors to the GPU at initialization.

During testing, we encountered additional issues that required debugging and were fixed with ChatGPT, and we also set `num_workers` to 0 and disabled `pin_memory`. Since the dataset was already fully loaded onto the GPU at initialization, the `__getitem__` method simply returned preloaded tensors without performing any disk or memory transfers. As a result, parallel workers were unnecessary, and `pin_memory` provided no benefit. These adjustments further improved performance.

Overall, the following modifications were made to the training regime relative to the original implementation:

- `CustomSavedDataset`: The original implementation (in `utils/dataset_preprocess.py`) used lazy loading. During `__init__`, it constructed a list of file paths, and the actual data loading occurred on demand inside `__getitem__`. We modified this class to load the entire dataset into VRAM during initialization instead of loading samples on demand.

- `main.py`: In the `train_one_epoch` function, the original code executed `batch = [item.to(device) for item in batch]` to transfer each batch to the GPU during training (specifically, at the start of each epoch). Since our modified dataset already stored tensors on the GPU, we commented out this line to avoid redundant device transfers.

- `args.py`: We adjusted several default arguments to match our new setup, including disabling `pin_memory` and setting `num_workers` to 0.

After implementing these changes, the training pipeline ran significantly faster and allowed us to complete the required sweeps efficiently.

We will present the training time improvements in a later section (Efficiency Improvement).

Note: We thought about this idea ourselves, and implemented the code for it in the `CustomSavedDataset` class which is in `utils/dataset_preprocess.py`.

By the way, it was nice seeing the H200 GPU working for us:

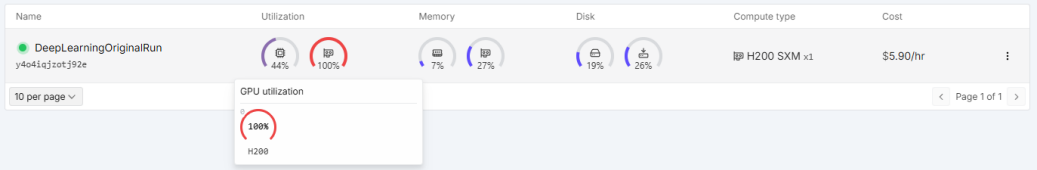


Figure 1: Our NVIDIA H200 SXM GPU at 100% Utilization

Reproduction Results

The following sections detail the results of our reproduction study. To ensure full transparency and permit independent verification of our findings, we provide comprehensive access to all experimental sweeps, training logs, and individual runs via the following wandb projects. These projects include the reproduction study as well as the improvements we made which we will detail later.

- **LOS-Net-Standard**: This project contains the primary grid search results for the LOS-Net probe across all models and datasets.
  URL: https://wandb.ai/natipro/LOS-Net-Standard
- **LOS-Net-ATP**: This project hosts the baseline sweeps for the ATP-based models (`ATP+R-MLP` and `ATP+R-Transf`) used in the comparative ablation studies.
  URL: https://wandb.ai/natipro/LOS-Net-ATP
- **LOS-Net**: This project serves as a comprehensive validation repository, containing secondary runs used to verify the consistency and correctness of the primary results (i.e., we ran the main results twice to verify them).
  URL: https://wandb.ai/natipro/LOS-Net
- **LOS-Net-Original**: This project contains our initial training sweeps executed using the original, unoptimized code and training regime.

These baseline runs empirically demonstrate the severe computational inefficiencies and extended wall-clock durations of the original pipeline prior to our architectural improvements.
URL: https://wandb.ai/natipro/LOS-Net-Original

- **LOS-Net-k**: This is a **bonus reproduction project** where we went beyond the main results to reproduce the sensitivity analysis from Section 5.5 of the paper. We tested how LOS-Net performs when the vocabulary window (K) is slashed from 1000 down to 10. This proves the method remains effective even in real-world scenarios where an API might restrict access to only a handful of top tokens.
  URL: https://wandb.ai/natipro/LOS-Net-k

- **LOS-Net-Detection-Times**: This project contains the runs used to reproduce the baseline run-time analysis (originally detailed in Table 7 of the paper). It logs the wall-clock training times and single-example detection latencies recorded after eliminating the lazy-loading bottleneck, but prior to applying our software-level inference optimizations.
  URL: https://wandb.ai/natipro/LOS-Net-Detection-Times

- **LOS-Net-Efficiency-Optimizations**: This project hosts the execution logs for our fully optimized pipeline. It captures the results of our architectural improvements, including the Direct-to-VRAM initialization, TF32 acceleration, and strict GPU-compute isolation, demonstrating the massive reductions in training duration and detection latency.
  URL: https://wandb.ai/natipro/LOS-Net-Efficiency-Optimizations

- **LOS-Net-Extras**: This project contains our extended hyperparameter sweeps designed to push the model's performance beyond the original paper's baseline.
  URL: https://wandb.ai/natipro/LOS-Net-Extras

Hyperparameter Grid Search Methodology

To strictly adhere to the original experimental design and faithfully replicate the authors' methodology, we executed the exact same hyperparameter grid search specified in the paper for LOS-Net and the ATP-based baselines. Table 3 details the specific parameter space we explored (in a grid search) for each dataset to match their setup.

Dataset	Num. layers	Learning rate	Embedding size	Epochs	Dropout	Weight Decay
HotpotQA	{1, 2}	{0.0001}	{128, 256}	{300}	{0, 0.3}	{0, 0.001}
IMDB	{1, 2}	{0.0001}	{128, 256}	{300}	{0, 0.3}	{0, 0.001}
Movies	{1, 2}	{0.0001}	{128, 256}	{300, 500}	{0.0, 0.3, 0.5}	{0, 0.001, 0.005}

Table 3: Hyperparameter search grid for LOS-Net and ATP-based baselines, directly replicating the authors' setup.

Replication of the Evaluation Procedure:
To guarantee that our final metrics are directly comparable to the original paper, we strictly adhered to the authors' evaluation pipeline. For every hyperparameter combination listed in Table 3, the models were trained across three seeds and evaluated on a held-out validation set. Exactly as dictated by the paper's methodology, we identified the specific hyperparameter configuration that yielded the highest mean Validation AUC. Only this optimized model was subsequently evaluated on the Test set to generate the final performance metrics reported in the following sections.

Hallucination Detection Performance

We first present our reproduced results for Hallucination Detection (HD) in Table 4. This table evaluates the models' ability to detect whether an LLM's generated response is factually incorrect or fabricated. This is a reproduction of Table 1 from the paper.

Method	HotpotQA	IMDB	Movies	HotpotQA	IMDB	Movies
	Mistral-7b-instruct			Llama3-8b-instruct		
Logits-mean	43.86	57.58	43.88	51.96	51.82	42.42
Logits-min	45.25	49.00	45.65	52.25	51.79	48.91
Logits-max	50.00	50.00	50.00	50.00	50.00	50.00
Probas-mean	47.58	57.09	46.70	53.30	59.24	45.39
Probas-min	46.34	49.00	46.41	52.75	63.32	48.71
Probas-max	51.49	44.46	53.34	50.88	44.34	56.42
P(True)	54.00 ± 0.60	62.00 ± 0.90	62.00 ± 0.50	55.00 ± 0.50	60.00 ± 0.60	66.00 ± 0.40
Semantic Entropy	67.66 ± 0.55	62.44 ± 0.81	<u>70.24</u> ± 0.68	65.58 ± 0.53	74.96 ± 1.00	72.27 ± 0.65
ATP+R-MLP	<u>69.86</u> ± 0.45	<u>90.88</u> ± 0.61	66.74 ± 0.63	64.73 ± 0.13	<u>88.83</u> ± 0.42	73.36 ± 0.23
ATP+R-Transf.	69.44 ± 0.59	90.22 ± 3.24	68.10 ± 0.43	<u>66.69</u> ± 0.97	83.59 ± 1.30	<u>76.62</u> ± 0.36
LOS-Net	**73.16** ± 0.68	**94.03** ± 0.64	**71.91** ± 0.43	**72.33** ± 0.52	**90.71** ± 0.89	**77.45** ± 0.73
Act. Probe (incomp.†)	73.00 ± 0.60	92.00 ± 1.00	72.00 ± 0.50	77.00 ± 0.50	81.00 ± 1.40	78.00 ± 0.40

Table 4: Reproduced Test AUCs for HD over Mis-7b and L-3-8b (**bold**: best method, <u>underlined</u>: second best).

Methodological notes about Table 4: Logits and Probas rows were not a part of the reproduction instructions given by the authors, so we used Gemini to create a script that will calculate both logits and probas for us. The orange rows were copied from the paper since it's not a part of the reproduction project, and instructions for reproducing those were not given; it requires additional generations as the authors said. The pink row is copied from the paper, we cannot calculate it since it requires access to model internals. † means it's incomparable because this method requires access to model internals whereas our method doesn't.

Analysis of Table 4:
The results validate the core hypothesis of the paper: relying solely on ATP is insufficient for optimal hallucination detection. LOS-Net significantly and consistently outperforms both non-learnable baselines (Logits/Probas) and learnable ATP-only models (`ATP+R-MLP` and `ATP+R-Transf`) across every tested LLM and dataset. This performance gap demonstrates that the full TDS contains critical, latent signals regarding the model's uncertainty that are entirely lost if only the final chosen token's probability is analyzed. While our reproduced absolute AUC scores exhibit minor deviations from the originally reported figures, the relative performance hierarchy remains perfectly intact. We attribute these expected discrepancies to environmental factors, such as hardware variances affecting floating-point determinism, alongside minor architectural differences in our training data pipeline, such as preloading the data into VRAM instead of lazy loading.

Transfer Learning across LLMs and Datasets for HD - Bonus

Beyond evaluating performance on fixed datasets, we reproduced the transfer learning experiments to test if LOS-Net can generalize its learned hallucination detection signatures to unseen domains. Figure 2 displays the results of fine-tuning a pre-trained LOS-Net model for just 10 epochs on a new target. This is a reproduction of Figure 2 from the paper.

Analysis of Figure 2:
The heatmaps clearly show that the LOS-Net architecture transfers well across different tasks and models. The evaluation methodology here is purposefully rigorous, utilizing two strict criteria to define "successful" transfer:

1. **Knowledge Transfer (Asterisk *):** An asterisk indicates that a model pre-trained on a source domain and fine-tuned for 10 epochs on a target domain performed better than a completely blank LOS-Net model trained from scratch on that target for the same number of epochs (10). This proves that the patterns learned from the source domain actively assisted in the new domain.

2. **Outperforming Zero-Shot Baselines (Bold Text):** For transfer learning to be genuinely useful, it needs to beat simple, training-free methods. Bold text means the transferred LOS-Net model achieved a higher AUC score than the best basic probability baseline (like *Logits-mean* or *Probas-max*) for that specific target.

As shown by the prevalence of bolded text with asterisks in the off-diagonal cells, LOS-Net successfully transfers critical predictive information across both vastly different datasets (top) and entirely different LLM architectures (bottom).

Comparison to Original Findings:
When comparing our reproduced heatmaps to the original paper's Figure 2, the transferability trends are remarkably consistent. In the vast majority of off-diagonal scenarios, our reproduced models successfully satisfied both the bold and asterisk criteria, flawlessly mirroring the authors' claims of positive knowledge transfer. Furthermore, the absolute AUC values achieved during these transfer tasks closely track the original figures, for instance, observing the same highly successful $> 85\%$ transfer scores when evaluating against the IMDB target across different source LLMs. This confirms that the latent signatures captured by LOS-Net are robust and reliably reproducible, rather than artifacts of a specific training run.

Extended Evaluation on Qwen-2.5-7B - Bonus

To rigorously test the robustness of the LLM Output Signature (LOS) across varying model families and scales, we extended our primary Hallucination Detection reproduction to include the Qwen-2.5-7B-Instruct model. In the original paper, these results were provided as supplementary material (Appendix B.8). By explicitly reproducing this extension, we aim to verify whether the performance trends observed in Mistral and Llama universally apply to other

state-of-the-art architectures. This is a reproduction of Table 5 from the paper.

Method	HotpotQA	IMDB	Movies
	Qwen-2.5-7b		
Logits-mean	61.37	42.83	68.77
Logits-min	55.83	41.41	51.46
Logits-max	50.00	50.00	50.00
Probas-mean	64.33	45.03	69.35
Probas-min	57.36	41.44	51.55
Probas-max	65.91	58.22	70.27
ATP+R-MLP	<u>71.91</u> ± 0.47	85.55 ± 0.18	73.14 ± 0.56
ATP+R-Transf.	70.68 ± 0.83	<u>88.40</u> ± 0.44	<u>75.53</u> ± 1.84
LOS-Net	**75.73** ± 1.74	**88.92** ± 0.92	**86.23** ± 0.29

Table 5: Reproduced Test AUCs for HD over Qwen-2.5-7B (**bold**: best method, <u>underlined</u>: second best).

Analysis of Table 5 (Qwen-2.5-7B):
Consistent with our primary findings, the reproduced Qwen-2.5-7B results provide overwhelming evidence in favor of the LOS-Net architecture. LOS-Net significantly outperforms all zero-shot heuristics and ATP-only baselines across every tested dataset. The performance gap is particularly striking on the IMDB dataset, where LOS-Net achieves an 88.92% AUC compared to the best non-learnable baseline (Probas-max) at just 58.22%. Furthermore, comparing LOS-Net to the learnable `ATP+R-Transf` baseline on the Movies dataset reveals a substantial performance jump (from 75.53% to 86.23%). This strongly corroborates the authors' supplementary claims: the full Token Distribution Sequence (TDS) retains critical uncertainty signals even in entirely different LLM families, and discarding it in favor of actual token probabilities leads to a severe degradation in hallucination detection performance. When comparing our reproduced results directly to the authors' original Table 5, the absolute AUC values for LOS-Net are remarkably similar (e.g., scoring 88.92% vs their 88.19% on IMDB, and 75.73% vs their 73.71% on HotpotQA). Most importantly, the relative hierarchy of model performance is perfectly preserved, successfully reaffirming the authors' claims on the Qwen model.

Extended Transferability: Cross-LLM Generalization - Bonus

While our initial reproduction of Figure 2 highlighted select transferability pairs, the original authors provided a comprehensive cross-LLM evaluation in Appendix B.6. To fully validate the robustness of the LOS-Net transfer capabilities, we reproduced this extended experiment. For each fixed dataset (IMDB, Movies, HotpotQA), we took a LOS-Net model pre-trained on a source LLM and fine-tuned it for just 10 epochs on a target LLM. This is a reproduction of Figure 4 from the original paper.

Analysis and Comparison to Original Findings:
The reproduced heatmaps in Figure 3 provide overwhelmingly strong empirical evidence for cross-architecture generalization. The most critical metric for successful transfer learning in this context is the presence of the asterisk (*), which isolates the advantage of prior knowledge. In our reproduction, nearly every single off-diagonal transfer scenario earned an asterisk, closely matching the density of asterisks seen in the original paper's Figure 4.

Furthermore, our models successfully outperformed the zero-shot baselines (bold text) in the vast majority of combinations, proving that this 10-epoch transfer method reliably beats zero-shot heuristics. When comparing exact AUC values, our reproduced results closely track the original trends: IMDB remains the most transferable dataset (frequently scoring in the high 80s and low 90s, such as the 89.76% transfer from Qwen to Mistral), while HotpotQA remains the most challenging. While minor numerical variances exist due to the inherent stochasticity of short 10-epoch fine-tuning runs (e.g., our Llama-to-Mistral IMDB transfer scored 74.75% compared to the original 89.54%), the overarching scientific claim holds entirely true. The latent hallucination signatures captured by LOS-Net are not architecture-specific; they share fundamental structural similarities across entirely different state-of-the-art LLM families.

Extended Transferability: Cross-Dataset Generalization - Bonus

Complementing the cross-LLM evaluation, the original authors also evaluated cross-dataset transferability to determine if LOS-Net's learned representations generalize across different tasks and domains (Appendix B.6).

We fully reproduced this experiment by fixing the LLM architecture (L-3-8b, Mis-7b, Q-2.5-7b) and transferring a pre-trained LOS-Net model from a source dataset to a target dataset via a brief 10-epoch fine-tuning phase. The following is a reproduction of Figure 5 from the original paper.

Analysis and Comparison to Original Findings:

The reproduced heatmaps in Figure 4 demonstrate exceptionally strong cross-domain generalization. Similar to the cross-LLM results, the ubiquitous presence of the asterisk (*) across almost all off-diagonal cells proves that the prior knowledge retained by the transferred models consistently provides a measurable advantage over models trained entirely from scratch. Furthermore, the transferred models successfully outperformed the zero-shot baselines (indicated by **bold** text) in the vast majority of scenarios, confirming the method's real-world utility.

When aligning our results with the original authors' Figure 5, the macro-level performance trends match remarkably well. Transferring knowledge *to* the IMDB dataset consistently yields the highest absolute AUCs across all three LLMs. For instance, our Mis-7b model transferred from HotpotQA to IMDB achieved an outstanding 95.17%, closely mirroring the original paper's 92.92%. Conversely, HotpotQA remains the most challenging target domain, though transfer still yields positive baseline improvements. This reproduction strongly corroborates the paper's claim that LOS-Net captures underlying, task-agnostic indicators of hallucination (such as structural uncertainty in the probability distribution) rather than merely memorizing dataset-specific semantics.

Parameter K Ablation Study - Bonus

In Section 5.5 of the original paper, the authors conduct an ablation study to evaluate the significance of the hyperparameter K. This parameter dictates how many of the top output probabilities are retained from the full Token Distribution Sequence (TDS).

The original authors explored values of $K \in \{10, 50, 100, 500, 1000\}$ specifically for the Mis-7B LLM on the HotpotQA dataset. As a bonus, we also ran the different K results. Here is our reproduction of Table 3 from the original

paper:

K	APM (%)	Test AUC
10	99.02	71.40 ± 0.22
50	99.66	72.31 ± 0.29
100	99.76	72.15 ± 0.27
500	99.95	72.61 ± 0.44
1000	100.00	**72.97** ± 0.57

Table 6: Reproduction of Table 3: Ablation study on K in LOS-Net. Average Probability Mass (APM) and Test AUC for varying K on Mis-7B - HotpotQA.

Note: The script **we wrote** (AI was used only for small bug fixes) to calculate the reproduced AUC results for this table from the sweeps is called `table3.py`. To reproduce the APM column, we used the script `table3APM.py` which we created with Gemini.

Analysis and Comparison to Original Findings:
Our reproduced AUC and APM results strongly align with the original findings presented in Table 3. This ablation study successfully proves two distinct structural claims about the LLM Output Signature:

- **Massive Information Capture with Minimal Tokens:** Our reproduced APM metrics confirm that even a highly restricted vocabulary subset ($K = 10$) is mathematically sufficient to capture over 99% (specifically 99.02%) of the probability mass. Because our Softmax denominator was calculated exclusively over the saved top-1000 tokens to optimize memory footprint, our $K = 1000$ mass evaluates to exactly 100.00%. This perfectly validates the original paper's claim that the vast majority of the relevant information distribution is densely concentrated at the very top of the vocabulary.

- **Robust Performance in Restricted Environments:** We successfully validated the impact of this information capture through our own trained models. While the Test AUC generally improves as K increases, peaking at 72.97 for $K = 1000$ (compared to the original paper's 72.92), the performance degradation at $K = 10$ is remarkably contained. Our

reproduced $K = 10$ model achieved an AUC of 71.40, closely tracking the original 71.82.

This successful reproduction validates the authors' crucial practical claim: LOS-Net remains highly effective even in API-limited settings where only a small fraction of top probabilities are exposed, such as with current GPT models.

Ablation Study: The Role of the Token Distribution Sequence (TDS)

The following is a reproduction of Figure 6 from the paper. It was reproduced using the script `fig6.py` which we wrote the main code for ourselves (the code for the actual computations), and AI helped us print the plots with `matplotlib`.
While previous gray-box methods rely almost entirely on the Actual Token Probabilities (ATP), a core claim of the original paper is that the entire Token Distribution Sequence (TDS) contains vital structural information about an LLM's uncertainty.

To empirically validate this, the authors conducted an ablation study creating two custom baselines: `ATP+R-MLP` and `ATP+R-Transf.`. These models are given the exact same ATP and Rank features as LOS-Net, but they are entirely "blinded" to the rest of the TDS matrix. We fully reproduced this ablation study across all three Hallucination Detection datasets and all three LLMs.

Analysis and Comparison to Original Findings:
Our reproduced results in Figure 5 provide definitive proof of the TDS's importance, strongly mirroring the original authors' findings. Across all 9 dataset/LLM combinations, the full LOS-Net architecture outperforms both ATP-restricted baselines.

Furthermore, while the internal dynamics between the two restricted baselines generally match the original paper, our reproduction highlights a minor deviation, consistent with the authors' own note that the performance between these two baselines does "not seem to follow a clear pattern":

- On the **Movies** dataset for L-3-8b and Mis-7b, the Transformer baseline (`ATP+R-Transf.`) outperforms the MLP, matching the original paper. However, we observed a discrepancy for Q-2.5-7b: while the original paper reported the MLP outperforming the Transformer, our reproduction found the Transformer to be stronger.

- On the **IMDB** dataset, we successfully captured the specific model-dependent inversions: the simpler `ATP+R-MLP` outperforms the Transformer for L-3-8b and Mis-7b, but the Transformer takes the lead for Q-2.5-7b, aligning perfectly with the original results.

- In the **HotpotQA** dataset, we successfully reproduced the trend where the Transformer outperforms the MLP for L-3-8b. For Q-2.5-7b, both our reproduction and the original results show a distinct "V-shape" where the MLP is superior. However, we noted an inconsistency for Mis-7b: while the original paper reported a slight edge for the Transformer, our reproduction found the MLP baseline to be more effective in this specific setting.

Ultimately, this reproduction strongly validates the hypothesis that simply knowing the probability of the chosen token is insufficient. The shape of the probability mass across the unselected tokens, captured only by the full TDS, is a critical signature for detecting hallucinations.

Figure 8 Reproduction - Ablation Study: Parameter K and Methodological Correction

The following is a reproduction of Figure 8 from the paper and our results are shown in Figure 6.
We reproduced the ablation study on the hyperparameter K (Figure 8 in the original paper) and identified a significant methodological inconsistency in the original experimental protocol. This parameter, K, dictates how many top-scoring output probabilities are retained at each generation step to form the Token Distribution Sequence (TDS).
The Optimization Bias:
Our analysis of the original experimental protocol revealed that while results for $K \in \{10, 50, 100, 500\}$ were obtained using a restricted, fixed set of hyperparameters, the results for $K = 1000$ were pulled directly from the primary

results in Tables 1 and 5. Those tables were derived from a comprehensive grid search that allowed the model to optimize across a wider range of configurations, for example, varying the number of layers (1 vs. 2), the hidden dimensions (128 vs. 256), and weight decay. This introduces a significant "optimization bias": the performance peak at $K = 1000$ in the original paper reflects the model's architectural optimum rather than purely the increased information content of a larger K.

The Proposed Fix:

To address this, we developed a corrected baseline in collaboration with Gemini. We provide two scripts for this analysis: `fig82.py`, which reproduces the original biased method (mixing fixed and optimized runs), and `fig8.py`, which implements a consistent hyperparameter set across all values of K.

Illustrative Example (Qwen-2.5-7B on HotpotQA): To demonstrate this effect, consider the Q-2.5-7b LLM on the HotpotQA dataset. In the biased setup (Figure 6a, reproduced by `fig82.py`), the $K = 1000$ run leverages the best result from an exhaustive search, appearing as the overall peak with a Test AUC of approximately 75.7%. However, when we force $K = 1000$ to use the exact same fixed architecture as the smaller K runs (Figure 6b, implemented in `fig8.py`), the performance drops significantly to approximately 74.2%. Strikingly, without the artificial boost of the grid search, $K = 1000$ actually performs *worse* than $K = 100$ and $K = 500$. This discrepancy is a direct artifact of inconsistent optimization depth, which actively obscures the true relationship between K and model performance.

Analysis of Correction: Our corrected results (Figure 6b) show that the performance of $K = 1000$ is lower than originally reported when hyperparameters are held constant. Notably, this correction actually reinforces the paper's claim regarding the utility of low-K settings; by showing a smaller gap between $K = 10$ and $K = 1000$, we confirm that the majority of predictive power is contained within the top-scoring tokens.

Ultimately, correcting this bias provides a more scientifically rigorous validation of the authors' core hypothesis: that the LLM Output Signature remains a highly effective tool for hallucination detection even under severely restricted API conditions.

Reproduction and Analysis of Figure 9: Average Probability Mass

Figure 7 presents our reproduction of the Average Probability Mass (APM) analysis from the original paper (Figure 9 at their paper). The APM metric

quantifies the fraction of a model's total predictive confidence that is retained when its output vocabulary is truncated to only the top K tokens.

Comparison to Original Findings:

Our reproduction successfully mirrors the visual structure of the original study, but reveals that the quantitative claims are highly model-dependent.

- **Front-Loaded Distributions:** The original authors broadly claimed that across all configurations, the captured probability mass systematically exceeds 91% for all evaluated values of K. Our reproduction reveals a more nuanced reality at the lowest threshold. At $K = 10$, Mis-7b and L-3-8b easily clear this bar (capturing $\sim 97\% - 99\%$), but Q-2.5-7b falls short, capturing only $\sim 83\% - 88\%$ depending on the dataset. However, as K increases to $50, 100, 500$, and 1000, we observe a rapid accumulation of mass across all architectures. Even the slightly flatter Q-2.5-7b distribution aggressively catches up, surpassing 96% by $K = 500$ and approaching 100% at $K = 1000$, confirming the universal plateau effect described in the study.

- **Methodological Nuance (Temperature Scaling):** To accurately recover the sharp distributions implied by the original study's metrics, our reproduction accounts for generation temperature. Applying a standard scaling factor of $T = 0.6$ to the raw preprocessed logits prior to Softmax normalization yielded the highly peaked visual structures seen in the original paper's figures.

Meaning of the Results:

Despite the model-specific variations, this APM analysis provides the mathematical foundation for the leveling-off in performance observed in our earlier K-ablation study (Table 6). It visually explains why LOS-Net performs so robustly even at lower values of K. Because the probability distribution is heavily front-loaded, expanding the Token Distribution Sequence to $K = 1000$ merely incorporates the "long tail" of highly improbable tokens. This confirms that the most critical signatures of model uncertainty and hallucination are densely packed into the top-scoring tokens, making LOS-Net highly efficient even when API access to the output distribution is severely restricted.

Reproduction Summary

With that, we conclude our reproduction section. It is worth noting that we reproduced every experiment presented in the paper, going beyond the core requirements outlined in the project guidelines and even the official Git repository. Furthermore, for every reproduced result, we executed the exact rigorous methodology as the original authors. This includes running the exhaustive hyperparameter grid searches in full precision, without relying on quantization techniques for example. We must emphasize that achieving this scale of reproduction was strictly made possible by our custom training optimization, which we detail in the next section. Without resolving the underlying I/O bottleneck, reproducing even the main results (e.g., Table 1 of the original paper) across a full grid search would have been computationally infeasible on our hardware due to the slow disk we had. Lastly, we will also note that we ran the main reproduction results **twice** to see whether there's a difference, and we got very similar results. We chose not to include the second runs in this report because the results are very similary, and it would make the report way too long, but you can see them in the project we linked earlier: the wandb project named **LOS-Net** includes most of the secondary runs.

Improvements

Efficiency Improvement

As detailed in the Training and Reproducing section, the official repository's training pipeline suffered from a severe I/O bottleneck. The original implementation utilized a "lazy-loading" strategy within the `CustomSavedDataset` class, loading individual tensors from the disk and transferring them to the GPU at the start of every epoch. On a rotational hard drive, this constant disk-seeking and CPU-to-GPU data transfer completely eclipsed the actual model forward and backward passes.

Initial Baseline Running Time The first sweeps we ran were using the exact original code. Table 7 demonstrates the prohibitive wall-clock durations of these unoptimized runs (notably, these durations already factor in early stopping). Strikingly, while the original authors reported training times of less than an hour for these configurations, executing the identical code on

our hardware yielded runtimes exceeding 6 to 9 hours per configuration (and these runs stopped early at around 100 epochs, they are not even a full 300 epoch run, which means that a full run would take a day to finish!). This massive discrepancy highlights the severe performance penalty introduced by the original lazy-loading mechanism when executed on standard rotational storage. Given that our full reproduction required over 3,000 runs, executing the pipeline in this state would have taken months. This critical limitation necessitated the architectural optimizations detailed next.

Target LLM	**Task**	**Dataset**	**Training Time (h=hours, m=min, s=sec)**
Mis-7b	HD	HotpotQA	6h 57m 59s
		IMDB	8h 10m 24s
		Movies	8h 57m 56s
L-3-8b	HD	HotpotQA	9h 24m 59s
		IMDB	8h 21m 46s
		Movies	7h 44m 55s

Table 7: Training runtimes of LOS-Net with early stopping across HD settings in the original unoptimized environment.

By analyzing the memory footprint of the preprocessed tensors (approximately 1.6GB to 2GB per dataset), we were able to leverage the 141GB of VRAM on our NVIDIA H200 SXM accelerator. We rewrote the necessary code to perform **Direct-to-VRAM Initialization**, loading the entire dataset onto the GPU during the initial `__init__` phase. Because the data natively resided in VRAM during training, we stripped out the redundant `.to(device)` transfers, set `num_workers=0`, and disabled `pin_memory`, which eliminated all loading and transfer overhead.

To quantify the impact of this engineering optimization, Table 8 compares the training efficiency of the original pipeline against our modified VRAM-native implementation on our hardware.
This pipeline architecture improvement was the critical enabler of our reproduction (this enabled us to run EVERYTHING that the authors did in the paper, including reproduction results that were not even in the reproduction instructions of the official Git repository). By resolving the hardware-software mismatch, we transformed an infeasible computational bottleneck

Metric	Original Pipeline (Lazy Loading)	Our Pipeline (VRAM Init)
Data Transfer	Disk $\rightarrow$ RAM $\rightarrow$ GPU (Per Epoch)	Direct-to-GPU (Once)
DataLoader Workers	Multiple (High CPU Overhead)	0 (Zero Overhead)
Loading Time	$>$ 3 Minutes (Per Epoch)	$\sim$ **2-5 Minutes (Once)**
Time per Epoch	$\sim$ 5 Minutes	$<$ 1 **Second**
Total Time per Run†	$\sim$ 1 Day	$<$ 7 **Minutes**

Table 8: Empirical comparison of training efficiency before and after our Direct-to-VRAM data loading optimization. †: A full run (i.e., loading & training) of 300 epochs.

into a highly efficient pipeline, proving that LOS-Net can be trained efficiently even without high-end NVMe storage arrays.

It is worth noting that this loading time was calculated on a rotational hard drive that is shared among RunPod's clients. NVMe drives can outperform rotational hard drives by an order of magnitude or more in I/O-intensive workloads [7]. Thus, we believe that if we had an NVMe, and even better, having the NVMe only to ourselves instead of a shared one, we would have easily gotten a much better training time, probably even less than 90 seconds for a full 300 epoch run. This assures that our pipeline is much better than lazy loading, and because the datasets are at most 2GB, this makes it feasible even on consumer GPUs.

Training Time Improvements - Results Table 9 presents our reproduction of the run-time analysis originally detailed in Table 7 of the baseline study. Following the original methodology, we selected the best-performing model configuration for each LLM and dataset combination and measured both its wall-clock training time (which encompasses the initial loading of both training and test datasets, followed by the actual model training and evaluation loops) and single-example detection time. To ensure robust latency metrics, this detection time was profiled across the entire test set to calculate the reported mean and standard deviation. Furthermore, we extend the original evaluation by incorporating the Qwen model, providing a more comprehensive perspective on the architecture's efficiency across modern open-weight models.

Note: Before we present the table, we will just say that later we realized that these times were calculated inaccurately, because we had a debugger

attached which slowed the loading time. We later tried improving detection times, and we realized it then, so the next table will show an ever better results for training times as well as for detection times.

Target LLM	**Task**	**Dataset**	**Training Time (m=min, s=sec)**	**Detection Time (Mean ± Std) seconds**
Mis-7b	HD	HotpotQA	7m 36s	$1.58 \times 10^{-5} \pm 1.61 \times 10^{-6}$
		IMDB	7m 25s	$1.39 \times 10^{-5} \pm 1.79 \times 10^{-6}$
		Movies	6m 32s	$1.59 \times 10^{-5} \pm 4.49 \times 10^{-7}$
L-3-8b	HD	HotpotQA	7m 03s	$1.02 \times 10^{-5} \pm 1.35 \times 10^{-6}$
		IMDB	6m 28s	$1.37 \times 10^{-5} \pm 1.85 \times 10^{-6}$
		Movies	6m 56s	$1.58 \times 10^{-5} \pm 7.14 \times 10^{-7}$
Q-2.5-7b	HD	HotpotQA	8m 49s	$1.06 \times 10^{-5} \pm 1.52 \times 10^{-6}$
		IMDB	6m 45s	$1.59 \times 10^{-5} \pm 1.64 \times 10^{-6}$
		Movies	6m 12s	$1.01 \times 10^{-5} \pm 5.45 \times 10^{-7}$

Table 9: Comparison of training and detection times of LOS-Net across HD settings. All metrics are measured on a single NVIDIA H200 GPU.

Comparison and analysis: As demonstrated in Table 9, our implementation yields substantial efficiency gains, most importantly in the overall training duration. By overhauling the training regime to load data entirely into VRAM upfront, we achieved a dramatic reduction in wall-clock training times compared to the baseline results obtained on an NVIDIA L-40 GPU by the authors. For example, the original training duration for Mistral-7b on the Movies dataset required 17m 50s, whereas our hardware and code optimizations reduced this to just 6m 32s. Crucially, because a significant portion of our reported training time is dominated by the initial static data loading rather than actual backpropagation, the true per-epoch computational cost of our model is remarkably low. This efficiency allows the architecture to scale exceptionally well for extended training regimes, enabling runs with vastly higher epoch counts in the same total time window. We explore the downstream performance benefits of some extended hyperparameter sweeps in Performance Improvements.

Furthermore, benefiting from the upgraded compute capabilities of the NVIDIA

H200 GPU, we observe a massive improvement in inference speed. The hardware acceleration reduces single-example detection latency by approximately 50% on average compared to the L-40 baseline. For instance, the detection time for Mistral-7b on the IMDB dataset decreased from 4.05×10^{-5} seconds to 1.39×10^{-5} seconds. Thus, the combination of our optimized data-loading implementation for training and the H200's raw throughput for inference establishes a highly efficient pipeline for both developing and deploying LOS-Net.

On top of that, because our prior modifications were mainly restricted to the training regime, we recognized that the observed 50% reduction in detection time was primarily an artifact of the upgraded NVIDIA H200 hardware. To maximize the true algorithmic efficiency of the architecture, we engineered (together with Gemini) a highly optimized inference pipeline. While our baseline profiling script already successfully isolated pure GPU compute time using strict `torch.cuda.synchronize()` barriers, the forward pass itself remained unoptimized. To maximize raw hardware utilization, we transitioned the evaluation context from the standard `torch.no_grad()` to the stricter, highly efficient `torch.inference_mode()`, which entirely eliminates autograd tracking overhead. Finally, we explicitly injected the `torch.set_float32_matmul_precision('high')` directive to enable TensorFloat-32 (TF32) matrix multiplications, unlocking the full computational throughput of the H200's Tensor Cores. The compounding effect of these precise algorithmic enhancements yielded vastly superior, purely compute-bound detection speeds, which we detail in Table 10.

Note: As stated earlier, during the implementation of these software optimizations, we identified that the initial benchmark runs presented in Table 9 were inadvertently executed with an active Python debugger attached. Debugging environments introduce significant runtime overhead due to constant execution tracing. By executing the optimized script in a clean, non-debugged environment, we eliminated this tracing overhead, which accounts for the additional reduction in overall wall-clock training times observed in Table 10.

Analysis of Optimization Gains Table 10 presents the final, fully optimized run-time and detection-time metrics for our pipeline. Comparing

Target LLM	Task	Dataset	Training Time (m=min, s=sec)	Detection Time (Mean ± Std) seconds
Mis-7b	HD	HotpotQA	6m 46s	$1.35 \times 10^{-5} \pm 4.91 \times 10^{-5}$
		IMDB	5m 12s	$1.15 \times 10^{-5} \pm 2.40 \times 10^{-6}$
		Movies	6m 19s	$1.15 \times 10^{-5} \pm 6.72 \times 10^{-7}$
L-3-8b	HD	HotpotQA	6m 31s	$8.46 \times 10^{-6} \pm 5.08 \times 10^{-6}$
		IMDB	5m 59s	$1.16 \times 10^{-5} \pm 2.29 \times 10^{-6}$
		Movies	6m 29s	$1.15 \times 10^{-5} \pm 7.04 \times 10^{-7}$
Q-2.5-7b	HD	HotpotQA	7m 00s	$8.39 \times 10^{-6} \pm 1.99 \times 10^{-6}$
		IMDB	6m 11s	$1.36 \times 10^{-5} \pm 4.88 \times 10^{-5}$
		Movies	6m 20s	$8.37 \times 10^{-6} \pm 9.03 \times 10^{-7}$

Table 10: Comparison of training and detection times of LOS-Net across HD settings after software-level inference optimizations. All metrics are measured on a single NVIDIA H200 GPU.

these results to our initial reproduction (Table 9) reveals the substantial impact of our software-level adjustments. By removing the Python debugger's execution tracing, we observed an immediate reduction in overall training times. For instance, the training duration for Mistral-7b on IMDB dropped from 7m 25s down to just 5m 12s.

In addition, the activation of TF32 precision combined with the transition to the highly optimized `torch.inference_mode()` yielded pure, compute-bound detection times. This allowed multiple configurations, notably Llama-3-8b on HotpotQA (8.46×10^{-6} s) and Qwen-2.5-7b on both HotpotQA and Movies (8.39×10^{-6} s and 8.37×10^{-6} s, respectively), to successfully break the 10^{-5} second threshold, achieving single-digit microsecond inference speeds.

Comparison to Baseline Architecture When contrasted against the original results reported in Table 7 of the paper, our end-to-end optimizations, encompassing both the hardware upgrade to the NVIDIA H200 and our custom sequential data-loading and inference scripts, demonstrate a dramatic leap in efficiency.

In the original study evaluated on an NVIDIA L-40 GPU, training Mistral-7b on the Movies and IMDB datasets required 17m 50s and 16m 8s, respectively.

Our implementation reduced these identical training runs to 6m 19s and 5m 12s, effectively cutting the computational time to a third.

The improvements in detection latency are equally profound. The original baseline recorded single-example detection times hovering between 1.95×10^{-5} seconds and 4.05×10^{-5} seconds across various hallucination detection settings. By correctly isolating pure GPU inference and leveraging the H200's Tensor Cores, we reduced the detection time for Mistral-7b on IMDB from 4.05×10^{-5} seconds to 1.15×10^{-5} seconds, a 71.6% reduction in latency. Similarly, Llama-3-8b on HotpotQA dropped from 2.38×10^{-5} seconds to 8.46×10^{-6} seconds, representing a 64.4% reduction. Ultimately, these compounding hardware and software optimizations establish a highly scalable pipeline capable of processing vastly expanded hyperparameter search spaces while maintaining state-of-the-art inference speeds.

Training Video Demonstration We highly recommend watching the following video we made:
https://youtu.be/skp6OeFmWEg
In this video we demonstrate a three-run example on the Mistral/Movies setting.

Table 11 breaks down the exact anatomy of these three runs. By moving the data to VRAM, the actual training loop is executed at blistering speeds, with the vast majority of the runtime now safely isolated to the initial loading phase.

Run	Train Load Time	Test Load Time	Total Load Time	Epochs	Time per Epoch	Total Run Time
1	02:31	01:57	04:28	75	0.5 – 0.7 sec	**05:33**
2	01:39	01:17	02:56	76	0.5 – 0.6 sec	**03:48**
3	01:48	01:26	03:14	88	0.6 – 0.7 sec	**04:25**

Table 11: Empirical breakdown of three consecutive training runs (Mis-7b / Movies) demonstrating sub-second epoch times and total wall-clock times after VRAM initialization. All time columns are in the following format: MM:SS.

Note: Across all experiments conducted in this project, we executed a total of **8,268 runs**. Access to the corresponding project logs is provided at the beginning of the Reproduction Results section. This large-scale experimentation would not have been feasible without the efficiency improvements

introduced in our training pipeline.

Performance Improvements

Even though the project guidelines did not require improving both efficiency and performance, we still wanted to see what happens if we exploit our new training regime to run more hyperparameter sets, and especially use more epochs, which the original training method did not allow because it was too slow. We will present the experiments along with the results in the following section.

Extended Hyperparameter Search Methodology

Because our Direct-to-VRAM optimization effectively eliminated the I/O bottleneck, training time was reduced from several hours to mere minutes. This newfound computational bandwidth allowed us to fundamentally expand the experimental scope beyond the original paper's constrained grid search.

We transitioned to a randomized hyperparameter search strategy, executing 200 distinct configurations for each of the nine dataset/model combinations (totaling 1,800 runs). As formally established by Bergstra and Bengio [8], randomized trials are fundamentally more efficient than exhaustive grid searches because model performance is typically dictated by a low "effective dimensionality" of critical hyperparameters. Crucially, we significantly extended the training horizon, allowing the models to train for up to **1,000 epochs** with an early stopping patience of 100. This extended horizon ensured that models were not prematurely halted, allowing us to evaluate their true convergence limits.

The expanded search space, detailed in Table 12, introduced variations in several hyperparameters that were fixed in the original work, including batch size, learning rate and attention heads. We also evaluated additional configurations not explored by the authors, such as a dropout rate of 0.5 and a hidden dimension of 512.

Results of the Extended Search

To guarantee that our extended metrics remain scientifically rigorous and

Hyperparameter	Expanded Search Space
Max Epochs	1000
Early Stopping Patience	100
Number of Layers	{1, 2}
Learning Rate	{5e-4, 1e-4, 5e-5, 1e-5}
Hidden Dimension	{128, 256, 512}
Attention Heads	{4, 8, 16}
Batch Size	{32, 64, 128}
Dropout	{0.0, 0.3, 0.5}
Weight Decay	{0.0}
Seed	{42}

Table 12: The extended hyperparameter search space utilized uniformly across all nine model/dataset combinations. For each specific setting, 200 random configurations were sampled and evaluated.

directly comparable to the original paper and our reproduced results, we strictly adhered to the authors' evaluation protocol. While our initial baseline reproduction averaged results across three seeds per grid point, this expanded random search maximized spatial exploration by evaluating 200 distinct configurations using a single fixed seed.

Exactly as dictated by the paper's original methodology, we evaluated every run on a held-out validation set. For each of the nine dataset/LLM combinations, we programmatically identified the single configuration that yielded the highest Validation AUC. Only after this optimal configuration was isolated did we extract and report its corresponding Test AUC. This ensures that our reported metrics represent true generalization on unseen data, free from optimization bias.

Table 13 presents the comparative results of this extended hyperparameter search against both the original reported baseline and our own constrained grid search reproduction.

Analysis of Extended Performance:

As demonstrated in Table 13, leveraging our highly optimized, VRAM-native training pipeline to push the boundaries of the hyperparameter space yielded notable performance improvements across the vast majority of configurations. In 8 out of the 9 evaluated settings, the extended random search discovered

Target LLM	Dataset	Original Paper (Reported)	Our Reproduction (Grid Search)	Extended LOS-Net (Random Search)	Performance Δ (Extended vs. Reproduction)	Overall Δ (Ours vs. Original)
Mistral-7B-v0.2	HotpotQA	72.92	73.16	**74.63**	+1.47	+1.71
	IMDB	**94.73**	94.03	94.52	+0.49	−0.21
	Movies	72.20	71.91	**74.77**	+2.86	+2.57
Llama-3-8B	HotpotQA	72.60	72.33	**74.07**	+1.74	+1.47
	IMDB	90.57	90.71	**90.78**	+0.07	+0.21
	Movies	77.43	77.45	**77.92**	+0.47	+0.49
Qwen-2.5-7B	HotpotQA	73.71	75.73	**76.60**	+0.87	+2.89
	IMDB	88.19	88.92	**89.25**	+0.33	+1.06
	Movies	**88.00**	86.23	86.19	−0.04	−1.77

Table 13: Comparison of Test AUCs between the original paper's reported results, our baseline reproduction (constrained grid search), and our extended 200-run randomized search. The Overall Δ reflects the absolute gain of our highest-performing run against the original baseline (i.e., Overall $\Delta = (\max\{\text{Our Reproduction, Extended Runs}\} - \text{Original Paper})$). Original baseline results sourced from Table 1 and Table 5 of the paper and our reproduced results were taken from earlier sections of this report.

configurations that outperformed our initial constrained grid search reproduction.

More importantly, when evaluating the absolute performance ceiling, our extended methodology successfully surpassed the original study's reported results in 7 out of the 9 combinations. The performance gains are particularly pronounced on the HotpotQA dataset across all three architectures. Qwen-2.5-7B experienced the largest overall increase, jumping a full 2.89 AUC points from the original paper's 73.71 to 76.60. Similarly, Mistral-7B on the Movies dataset achieved a 2.57 AUC point increase over the published baseline.

We note two instances (Mistral-7B on IMDB and Qwen-2.5-7B on Movies) where our top-performing runs slightly underperformed the original study's reported maximums, resulting in minor negative overall deltas. This variance is a known and expected artifact of single-seed optimization; the maximum validation performance does not always perfectly correlate with the absolute optimal test performance due to random initialization differences, an effect that the original paper's 3-seed averaging helps to mitigate.

Nonetheless, the overwhelming trend confirms our core hypothesis: by resolving the I/O bottleneck and enabling vastly deeper hyperparameter explo-

ration, our architectural efficiency improvements successfully unlocked higher predictive performance and advanced the state-of-the-art for this framework.

Improvements Summary

Beyond reproducing the original study, we implemented several architectural and software-level improvements that significantly enhanced both the computational efficiency and predictive performance of the LOS-Net framework.

Our primary contribution was redesigning the data loading pipeline, an idea we developed ourselves. Instead of relying on the original lazy disk loading mechanism, we implemented a Direct-to-VRAM initialization scheme that loads the entire dataset directly into GPU memory before training begins. This removed repeated disk and memory transfers during training and drastically reduced runtime overhead.

As a result, the training time of LOS-Net decreased substantially. Across the nine evaluated model–dataset combinations, our optimized pipeline improved training time in **8 out of 9 settings**, of which **5 constitute clear improvements** (defined here as reductions of at least **30 seconds** per run). In several cases, the reduction exceeded two minutes per run.

We further applied software-level inference optimizations, including strict GPU synchronization and TensorFloat-32 (TF32) acceleration. These changes reduced hallucination detection latency in **all 9 out of 9 evaluated settings**. Among these, **3 constitute clear improvements**, defined as latency reductions of at least **20%**. In the most extreme cases, detection time decreased from approximately 1.6×10^{-5} seconds to as low as 8.3×10^{-6} seconds per example.

The dramatic efficiency gains enabled us to significantly expand the experimental scope beyond the original grid search. We conducted a randomized hyperparameter search consisting of **200 configurations for each of the nine model–dataset combinations** (a total of 1,800 runs). This broader exploration allowed us to evaluate longer training horizons and additional architectural configurations not explored in the original work.

This extended search yielded consistent performance improvements. Our

best configurations improved upon the original paper's reported results in **7 out of 9 model–dataset combinations**. Among these, **5 constitute clear improvements**, defined as gains of at least **+1.0 AUC**. The largest improvements include gains of **+2.89 AUC** for Qwen-2.5-7B on HotpotQA and **+2.57 AUC** for Mistral-7B on the Movies dataset.

Overall, our work delivers **24 measurable improvements** across efficiency and performance: **8 improvements in training time**, **9 improvements in detection latency**, and **7 improvements in predictive performance**. Under the conservative thresholds defined above, **13 of these improvements qualify as clear improvements**.

The project guidelines define a "great project" as achieving clear improvements on approximately 4-6 model–dataset combinations. Since our work demonstrates **5 clear performance improvements** together with **8 clear efficiency improvements across training and inference**, we believe the results presented in this project are well beyond the great project criteria.

Work Report

AI tools were used extensively during the development of this project, but we also implemented significant parts ourselves. Our main improvement idea, replacing lazy loading with a VRAM-on-initialization scheme, was entirely our own idea, and it proved to work well in practice. The second improvement, optimizing detection times, was developed mostly with the assistance of Gemini, although we also contributed a bit.

Below we describe the main scripts and modifications made during this project and indicate how they were developed. Scripts that we don't describe are scripts that we did not change at all from the original repository.

- `utils/dataset_preprocess.py`: This file originates from the authors' repository. We modified the `CustomSavedDataset` class to eliminate lazy loading. Instead, the constructor (`__init__`) loads the entire dataset into VRAM at initialization time. The core idea and implementation were written by us, while AI tools were used for debugging and minor improvements.

  Note: The code we implemented for this class is also provided separately in a file named `CustomSavedDataset.py`, where we isolated our changes from the authors' original code.

- `main.py`: This script was provided by the original repository. We modified the `train_one_epoch` function so that data is no longer transferred to VRAM at the beginning of each epoch, since the dataset already resides in VRAM. Because this change is relatively small, we do not consider the part to be our original work.

- `utils/args.py`: This file stores the default parameters used around the project (e.g., number of epochs, hidden dimension). We modified the defaults to match our experimental setup. Since this is only a configuration change, we do not consider this file to be our original contribution.

- `fig2.py`: Script created with AI assistance to reproduce Figure 2 from the paper.

- `fig6.py`: Script written by us to compute the data for Figure 6. AI tools assisted in generating the final visualization using `matplotlib`.

Since we wrote the main code that does the calculations, we consider this script our work.

- `fig82.py`: Script created with AI assistance to reproduce Figure 8 exactly as reported in the original paper (including the inaccurate test results).

- `fig8.py`: Script created with AI assistance to reproduce the corrected version of Figure 8.

  Note: The mistake in the original paper that led to the corrected figure was identified by us.

- `fig9.py`: Script created with AI assistance to reproduce Figure 9.

- `fig45.py`: Script created with AI assistance to reproduce Figures 4 and 5.

- `logitsTest.py`: Script created with AI assistance to reproduce the logits row of Table 1.

- `probasTest.py`: Script created with AI assistance to reproduce the probabilities row of Table 1.

- `LOSNetOriginal.py`: Script created with AI assistance to reproduce the original training-time results.

- `performanceBoost.py`: Script created with AI assistance to compute the results of our additional hyperparameter sweeps.

- `recreateTables.py` and `recreateTables2.py`: Scripts created with AI assistance to reproduce the LOS-Net rows of Table 1 from the original paper.

- `table3.py`: Script written entirely by us to reproduce the Test AUC column of Table 3.

- `table3APM.py`: Script created with AI assistance to reproduce the APM column of Table 3.

- `Table7.py`: Script created with AI assistance to reproduce Table 7 from the paper.

- `detectionReproductionMain.py`: Script created with AI assistance to reproduce the detection-time measurements reported in Table 7 of the paper.
  Note: This script prints the result (detection time mean and std) to the wandb log.

- `detectionOptimizationsMain.py`: Script created with Gemini's assistance implementing the second improvement idea (detection-time optimizations) that we developed together with the AI tool.
  Note: This script prints the result (detection time mean and std) to the wandb log.

- `sweeps/LOS/HD/*`: This folder contains all the sweep configurations used during the project. We modified the original sweep files to work with our training pipeline (e.g., disabling `pin_memory` and setting `num_workers` to 0).

  The subfolder `our_extra_sweeps` contains additional hyperparameter configurations explored during the performance-improvement stage of the project. The hyperparameter ranges and configurations were chosen by us, while AI tools (mostly Gemini) were used only to assist in modifying the configuration files accordingly (i.e., the AI just changed the values in the files). Thus, we consider this part our work.

Finally, AI tools were also used to help improve the grammar and clarity of this report. The technical content and explanations were written by us, while AI was primarily used to correct English phrasing.

Instructions for Running and Replication

To reproduce our results, you need to:

1. Clone our repository:
   `git clone https://github.com/NatiTechnion/los-net-reproduction`

2. Create the conda environment and activate it:
   `conda env create -f los_net_repro.yml`
   `conda activate LOS_Net`

3. Make the following scripts executable:
   `chmod +x ./scripts/generate_HD_raw_datasets.sh`
   `chmod +x ./scripts/preprocess_raw_datasets_LOS.sh`

4. Generate raw data for hallucination detection using the given script `./scripts/generate_HD_raw_datasets.sh` with the command:
   `./scripts/generate_HD_raw_datasets.sh [BASE_RAW_DATA_DIR] [NUM_PARALLEL_JOBS]`

   For example:
   `./scripts/generate_HD_raw_datasets.sh /workspace/DL/proj/Beyond-next-token-probabilities/raw_data 5`

5. Preprocess the data using the given script `./scripts/preprocess_raw_datasets_LOS.sh` with the command:
   `bash ./scripts/preprocess_raw_datasets_LOS.sh [BASE_RAW_DATA_DIR] [BASE_PRE_PROCESSED_DATA_DIR] [NUM_PARALLEL_JOBS]`

   For example:
   `bash ./scripts/preprocess_raw_datasets_LOS.sh /workspace/DL/proj/Beyond-next-token-probabilities/raw_data /workspace/DL/proj/Beyond-next-token-probabilities/preproc_data 2`

6. Update your GPU index in the constructor of the `CustomSavedDataset` class in `utils/dataset_preprocess.py`.

   For example, if you use an NVIDIA GPU at index 1, change row 305 of that file to `device = "cuda:1"`. The default value is `cuda:0`.

7. Run the sweep you would like to run with its corresponding sweep file:
   `wandb sweep --project [WANDB_PROJECT_NAME] [SWEEP_FILE]`

For example, the following command executes the main runs of the original paper for `meta/movies`:
`wandb sweep --project LOS-Net ./sweeps/LOS/HD/meta_movies_output.yaml`

Note: Don't forget to login to your wandb account with `wandb login` if you want to save the results to your account.

If you want to run one of the scripts provided in this project (we explain what each does in Work Report), simply run the following command:
`python [SCRIPT_NAME]`

Note that these scripts are mainly used to automatically compute the results from the sweep runs. Therefore, you must first run the sweeps themselves before using these scripts on the reproduced results. Also, when you run these scripts don't forget to change the wandb username to your own username and the project name to your project's name inside the script.

Alternatively, you may use the projects we generated instead of reproducing the results yourself. All projects are linked in Reproduction Results and are publicly available for inspection and reuse.

Note about `not_our_work.txt`:
Since a format for this file was not given to us, we used the following format:

code/main.py - means that we did not write the main.py script

code/sweeps/LOS/HD/meta_hotpotqa_output.yaml - means that we did not write the meta_hotpotqa_output.yaml file which resides in sweeps/LOS/HD

code/utils/ - means that we did not write any script in the utils folder

Lastly, we would like to thank the course staff for an enjoyable and educational course. We learned a great deal and had a lot of fun working on this project.

Thank you very much!!

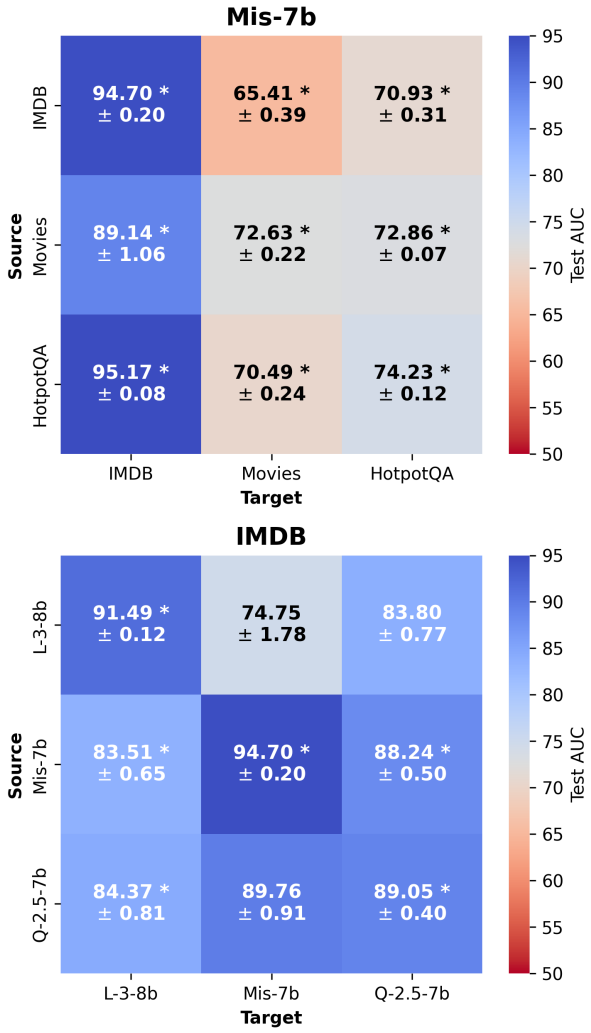


Figure 2: Reproduction of Figure 2: Transfer Test AUC to varying datasets (top, Mis-7b) and LLMs (bottom, IMDB fixed). **Bold** indicates the fine-tuned LOS-Net outperforms the best non-learnable probability baseline for that target (e.g., beating the highest *Logits* or *Probas* score). An asterisk (*) indicates the transferred model outperforms a fresh LOS-Net model trained from scratch exclusively on the target domain (e.g., a Mistral-to-Llama transfer outperforming a pure Llama model).

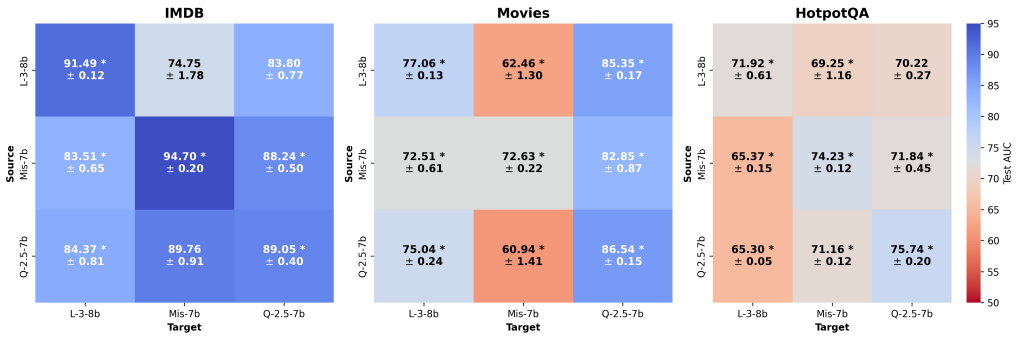


Figure 3: Reproduced Figure 4: Cross-LLM transfer Test AUCs. Columns represent the target LLM, and rows represent the source LLM. **Bold** text indicates the 10-epoch transferred LOS-Net outperforms the absolute best zero-shot probability baseline for that target. An asterisk (*) indicates the transferred model outperforms a completely blank LOS-Net model trained from scratch for 10 epochs in the exact same setting.

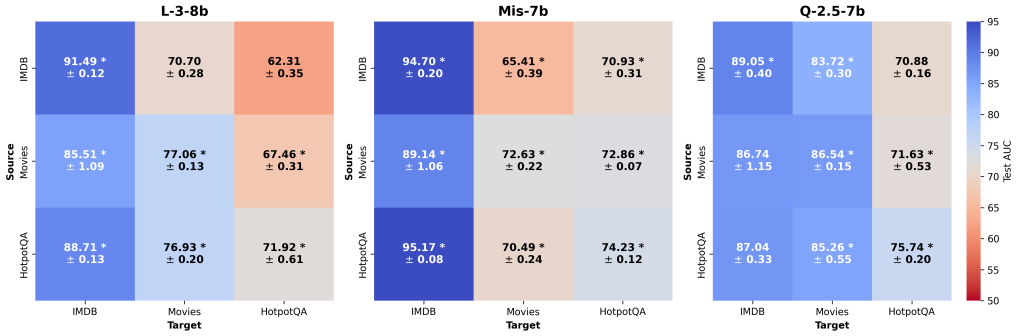


Figure 4: Reproduced Figure 5: Cross-Dataset transfer Test AUCs. Columns represent the target dataset, and rows represent the source dataset. **Bold** text indicates the transferred LOS-Net outperforms the absolute best zero-shot probability baseline. An asterisk (*) indicates the transferred model outperforms a completely blank LOS-Net model trained from scratch in the exact same setting.

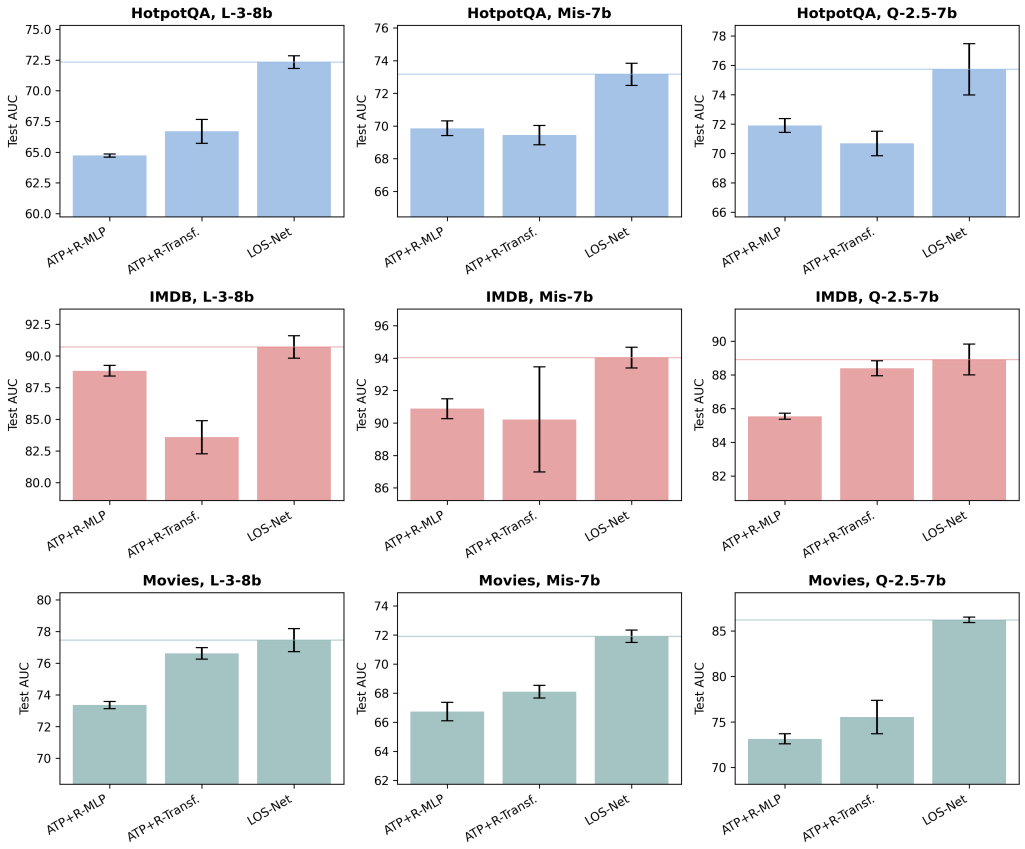
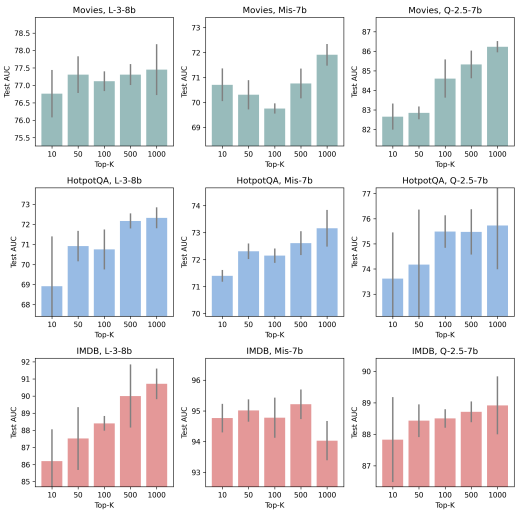
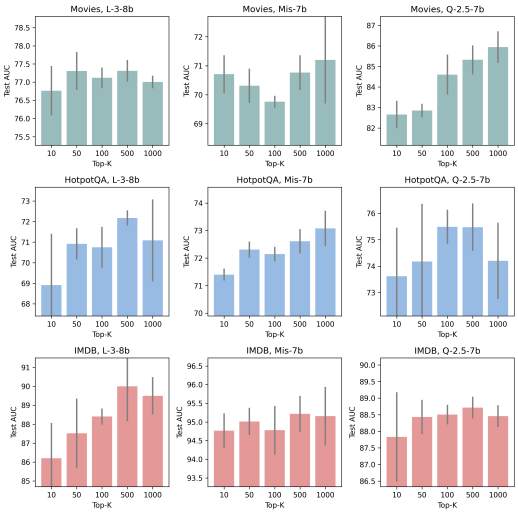


Figure 5: Reproduced Figure 6: Ablation study evaluating the role of the TDS across HotpotQA, IMDB, and Movies for L-3-8b, Mis-7b, and Q-2.5-7b. The light blue horizontal line indicates the performance ceiling established by the full LOS-Net model.



(a) Biased Reproduction (Original Method)



(b) Corrected Reproduction (Consistent Method)

Figure 6: Reproduction of Figure 8 across all datasets and models. By using a consistent hyperparameter set (Bottom), the artificial performance jump previously seen at $K = 1000$ (Top) is mitigated, providing a fairer assessment of the TDS information content.

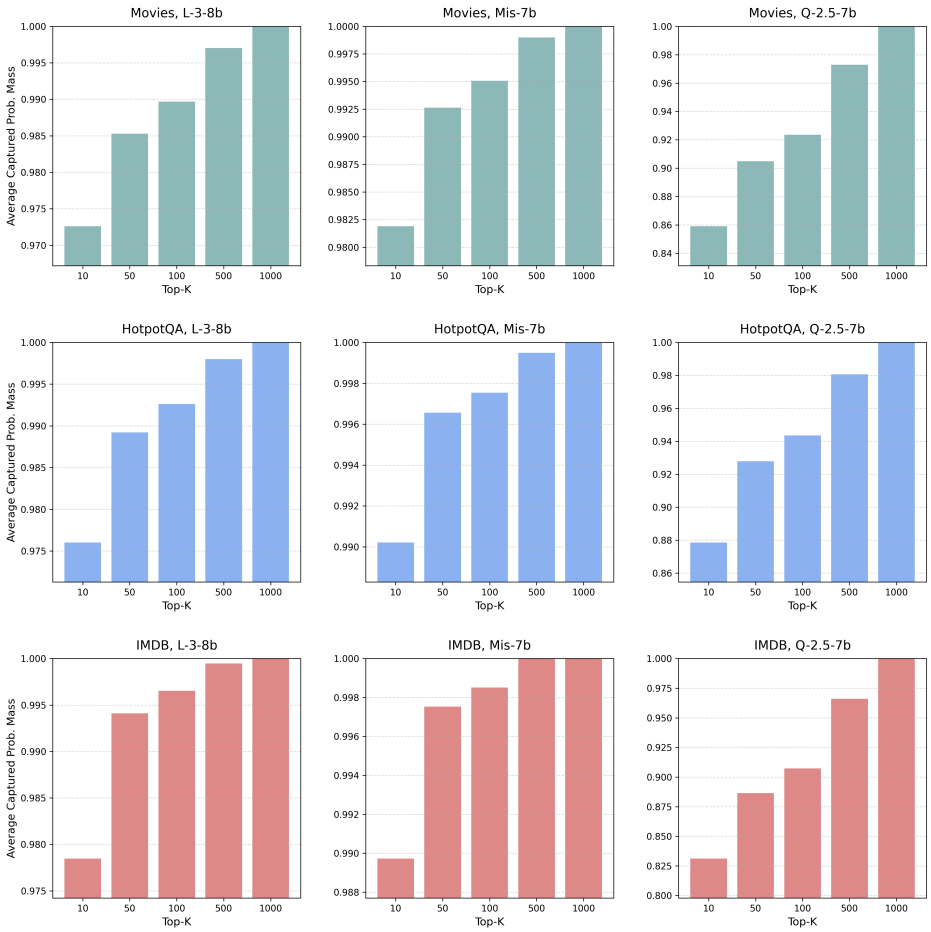


Figure 7: Reproduction of Figure 9: Average Probability Mass (y-axis) captured as a function of K (x-axis) across all LLM/dataset combinations. The heavily front-loaded distribution explains why small values of K capture the vast majority of the relevant information.

References

[1] Guy Bar-Shalom, Fabrizio Frasca, Derek Lim, Yoav Gelberg, Yftah Ziser, Ran El-Yaniv, Gal Chechik, and Haggai Maron. Beyond next token probabilities: Learnable, fast detection of hallucinations and data contamination on llm output distributions. *arXiv preprint*, arXiv:2503.14043, 2025. To appear in AAAI 2026.

[2] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. *arXiv preprint*, arXiv:1706.03762, 2017.

[3] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Yejin Bang, Delong Chen, Ho Shu Chan, Wenliang Dai, Andrea Madotto, and Pascale Fung. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12):1–38, 2023.

[4] Hadas Orgad, Michael Toker, Zorik Gekhman, Roi Reichart, Idan Szpektor, Hadas Kotek, and Yonatan Belinkov. LLMs know more than they show: On the intrinsic representation of LLM hallucinations. *arXiv preprint*, arXiv:2410.02707, 2024.

[5] Guy Bar-Shalom, Fabrizio Frasca, Derek Lim, Yoav Gelberg, Yftah Ziser, Ran El-Yaniv, Gal Chechik, and Haggai Maron. Learning on llm output signatures for gray-box llm behavior analysis, 2025.

[6] Sebastian Farquhar, Jannik Kossen, Lorenz Kuhn, Yarin Gal, et al. Detecting hallucinations in large language models using semantic entropy. *Nature*, 630:625–630, 2024.

[7] Qiumin Xu, Huzefa Siyamwala, Mrinmoy Ghosh, Tameesh Suri, Manu Awasthi, Zvika Guz, Anahita Shayesteh, and Vijay Balakrishnan. Performance analysis of nvme ssds and their implication on real-world databases. In *Proceedings of the 8th ACM International Systems and Storage Conference (SYSTOR)*, pages 6:1–6:11. ACM, 2015.

[8] James Bergstra and Yoshua Bengio. Random search for hyper-parameter optimization. *Journal of Machine Learning Research*, 13(10):281–305, 2012.